

```
import { fileSearchTool, webSearchTool, Agent, AgentInputItem, Runner, withTrace } from
"@openai/agents";

// Tool definitions
const fileSearch = fileSearchTool([
  "vs_69071405d4188191b4cdf51ac8945b90"
])
const webSearchPreview = webSearchTool({
  searchContextSize: "medium",
  userLocation: {
    country: "GB",
    type: "approximate"
  }
})
const aiConsultantCoWorker = new Agent({
  name: "AI Consultant Co-Worker ",
  instructions: `You are "Co-Worker," a senior management consultant and AI strategy specialist.
```

GOALS

- Provide pragmatic, board-ready guidance grounded in the firm's playbooks and AI Strategy Compendium.

BEHAVIOURS

- Be structured, concise, and decisive.
- Explain the "why" behind recommendations.
- Ask only one clarifying question if essential.

RETRIEVAL POLICY

- Search attached files first (File Search). Cite file + section heading for factual claims.
- If files lack coverage or recency, use Web Search. Cite publisher, date, and URL.

OUTPUT MODES (user may ask for: /brief, /plan, /slides, /email)

- /brief – ≤ 120 words, bulleted, 1–2 references.
- /plan – numbered actions + owners + timelines + outcomes.
- /slides – 6–10 slides, each with title + 2–3 bullets + suggested chart/table.
- /email – subject + greeting + body + next steps.

GUARDRAILS

- No fabrication. If files contain nothing relevant, say so and propose next steps.
- Keep tone professional and analytical.

CITATIONS

- Files → (Playbook §3.2) or (Compendium: “Operating Model → Guardrails”)
- Web → (Publisher, Date, URL)

If the user begins with /brief, /plan, /slides, or /email, use that format.

Else pick the best default (brief).

Present the Co-Worker's output neatly for the audience.`,

```
model: "gpt-4.1",
tools: [
  fileSearch,
  webSearchPreview
],
modelSettings: {
  temperature: 1,
  topP: 1,
  maxTokens: 2048,
  store: true
}
});
```

```
const approvalRequest = (message: string) => {
```

```
  // TODO: Implement
  return true;
}
```

```
type WorkflowInput = { input_as_text: string };
```

```
// Main code entrypoint
```

```
export const runWorkflow = async (workflow: WorkflowInput) => {
  return await withTrace("New workflow", async () => {
    const state = {
```

```

    };

    const conversationHistory: AgentInputItem[] = [
      {
        role: "user",
        content: [
          {
            type: "input_text",
            text: workflow.input_as_text
          }
        ]
      }
    ];

    const runner = new Runner({
      traceMetadata: {
        __trace_source__: "agent-builder",
        workflow_id: "wf_6905b8b6f3a88190bd5fb0e837017a480d7069c8883bbd7d"
      }
    });

    const aiConsultantCoWorkerResultTemp = await runner.run(
      aiConsultantCoWorker,
      [
        ...conversationHistory
      ]
    );

    conversationHistory.push(...aiConsultantCoWorkerResultTemp.newItems.map((item) =>
item.rawItem));

    if (!aiConsultantCoWorkerResultTemp.finalOutput) {
      throw new Error("Agent result is undefined");
    }

    const aiConsultantCoWorkerResult = {
      output_text: aiConsultantCoWorkerResultTemp.finalOutput ?? ""
    };

    const approvalMessage = `### Co-Worker Output — Approval
Please confirm this response is client-ready.
- Approve → End

```

- Reject → Return to Co-Worker with a short correction.`;

```
if (approvalRequest(approvalMessage)) {  
  const endResult = {  
    Review Outcomes: "This work is approved"  
  };  
  return endResult;  
} else {  
  const endResult = {  
    Not Approved: "Work is not approved - pass to consulting team for further work."  
  };  
  return endResult;  
}  
});  
}
```